

Spread Rest JavaScript

Today 1:09 AM



tell me about spread and rest in details with example in javascript

Absolutely. **Spread (...)** and **Rest (...)** are two very important JavaScript features. They use the **same three dots (...)**, but their purpose depends on **where they are used**.

1. Spread Operator (...)

The **spread operator** is used to **expand/unpack** elements of an iterable (like an array or string) or properties of an object.

Think:

Spread = take things OUT

Spread with Arrays

Basic example

```
JavaScript
const numbers = [10, 20, 30];
console.log(...numbers);
```

Output:

```
10 20 30
```

Instead of:

```
JavaScript
console.log(numbers);
```

which gives:

```
[10, 20, 30]
```

`...numbers` removes the array wrapper and spreads its individual values.

2. Copying an Array

One of the most common uses of spread is copying an array.

```
JavaScript
const original = [10, 20, 30];

const copy = [...original];

console.log(copy);
```

Output:

```
[10, 20, 30]
```

Now:

```
JavaScript
copy.push(40);

console.log(copy);
console.log(original);
```

Output:

```
[10, 20, 30, 40]
[10, 20, 30]
```

The arrays are separate.

Why not just do this?

```
JavaScript
const copy = original;
```

Because this does **not** create a new array.

```
JavaScript
const original = [10, 20, 30];
const copy = original;

copy.push(40);

console.log(original);
```

Output:

```
[10, 20, 30, 40]
```

Both variables point to the same array.

3. Combining Arrays

Spread makes combining arrays very easy.

```
JavaScript
const frontend = ["HTML", "CSS", "JavaScript"];
const backend = ["Node.js", "Express", "MongoDB"];
```

```
const skills = [...frontend, ...backend];
console.log(skills);
```

Output:

```
JavaScript
[
  "HTML",
  "CSS",
  "JavaScript",
  "Node.js",
  "Express",
  "MongoDB"
]
```

You can also add values while spreading:

```
JavaScript
const skills = [
  "Git",
  ...frontend,
  "React",
  ...backend
];
console.log(skills);
```

Output:

```
JavaScript
[
  "Git",
  "HTML",
  "CSS",
  "JavaScript",
  "React",
  "Node.js",
  "Express",
  "MongoDB"
]
```

4. Spread with Function Arguments

Suppose you have:

```
JavaScript
const numbers = [10, 20, 30];
```

And a function:

```
JavaScript
function add(a, b, c) {
  return a + b + c;
}
```

You can do:

```
JavaScript
console.log(add(...numbers));
```

It's equivalent to:

```
JavaScript
console.log(add(10, 20, 30));
```

Output:

```
60
```

This is extremely useful when you have an array but a function expects individual arguments.

5. Finding Maximum with Spread

Consider:

```
JavaScript
const numbers = [10, 50, 30, 90, 20];
console.log(Math.max(...numbers));
```

Output:

```
90
```

Without spread:

```
JavaScript
Math.max(numbers);
```

wouldn't work as intended because `Math.max()` expects individual numeric arguments.

Spread converts:

```
JavaScript
[10, 50, 30, 90, 20]
```

into:

```
JavaScript
10, 50, 30, 90, 20
```

6. Spread with Strings

Strings are iterable, so you can spread them.

```
JavaScript
```

```
const name = "Navi";
console.log(...name);
```

Output:

```
JavaScript
["N", "a", "v", "i"]
```

You can also use:

```
JavaScript
console.log(...name);
```

Output:

```
N a v i
```

7. Spread with Objects

Spread also works with objects.

```
JavaScript
const user = {
  name: "Navi",
  age: 22
};

const copy = {
  ...user
};

console.log(copy);
```

Output:

```
JavaScript
{
  name: "Navi",
  age: 22
}
```

This creates a new object.

8. Combining Objects

This is one of the **most important uses** of object spread.

```
JavaScript
const personalInfo = {
  name: "Navi",
  age: 22
};

const professionalInfo = {
```

```

    role: "Developer",
    experience: "Fresher"
  };

  const user = {
    ...personalInfo,
    ...professionalInfo
  };

  console.log(user);

```

Output:

```

JavaScript
{
  name: "Navi",
  age: 22,
  role: "Developer",
  experience: "Fresher"
}

```

9. Updating Object Properties

Spread is commonly used to create an updated copy of an object.

```

JavaScript
const user = {
  name: "Navi",
  age: 22,
  city: "Delhi"
};

const updatedUser = {
  ...user,
  age: 23
};

console.log(updatedUser);

```

Output:

```

JavaScript
{
  name: "Navi",
  age: 23,
  city: "Delhi"
}

```

Notice:

```

JavaScript
...user,
age: 23

```

The later **age: 23** overwrites the previous **age**.

10. Property Order Matters

Consider:

```
JavaScript
const user = {
  name: "Navi",
  age: 22
};

const updatedUser = {
  age: 30,
  ...user
};

console.log(updatedUser);
```

Output:

```
JavaScript
{
  age: 22,
  name: "Navi"
}
```

Why?

Because:

```
JavaScript
...user
```

comes **after** age: 30.

So the value from `user` wins.

Rule

For duplicate properties:

The last value wins.

```
JavaScript
const user = {
  name: "Navi",
  age: 22
};

const result = {
  ...user,
  age: 25
};
```

Result:

```
JavaScript
{
  name: "Navi",
  age: 25
}
```

11. Spread vs Rest

Now comes the important part.

Both use:

```
JavaScript
...
```

But they do opposite things.

Spread

Expands values.

```
JavaScript
const numbers = [10, 20, 30];
console.log(...numbers);
```

Think:

```
[10, 20, 30]
  ↓
10 20 30
```

Rest

Collects values.

```
JavaScript
function add(...numbers) {
  console.log(numbers);
}
add(10, 20, 30);
```

Output:

```
JavaScript
[10, 20, 30]
```

Think:

```
10 20 30
  ↓
[10, 20, 30]
```

So:

Spread = unpack

Rest = pack

12. Rest Parameters

Rest is commonly used in functions.

```
JavaScript
function add(...numbers) {
  console.log(numbers);
}

add(10, 20, 30, 40);
```

Output:

```
JavaScript
[10, 20, 30, 40]
```

The function can accept any number of arguments.

Then we can use array methods:

```
JavaScript
function add(...numbers) {
  return numbers.reduce((sum, num) => sum + num, 0);
}

console.log(add(10, 20, 30));
```

Output:

```
60
```

And:

```
JavaScript
console.log(add(10, 20, 30, 40, 50));
```

Output:

```
150
```

13. Normal Parameters + Rest

You can combine regular parameters with rest.

```
JavaScript
function introduce(name, age, ...skills) {
  console.log(name);
  console.log(age);
  console.log(skills);
}

introduce(
  "Navi",
  22,
  "JavaScript",
  "React",
```

```
"Node.js",
"MongoDB"
);
```

Output:

```
Navi
22
["JavaScript", "React", "Node.js", "MongoDB"]
```

Here:

```
JavaScript
name
```

gets:

```
"Navi"
```

age gets:

```
22
```

And:

```
JavaScript
...skills
```

collects the remaining arguments:

```
JavaScript
["JavaScript", "React", "Node.js", "MongoDB"]
```

14. Rest Must Be Last

This is valid:

```
JavaScript
function test(a, b, ...rest) {
  console.log(a);
  console.log(b);
  console.log(rest);
}
```

But this is invalid:

```
JavaScript
function test(...rest, a, b) {
}
```

Because rest must collect the **remaining values**, so it must be at the end.

15. Rest in Array Destructuring

Rest isn't limited to functions.

You can use it with array destructuring.

```
JavaScript
const numbers = [10, 20, 30, 40, 50];

const [first, ...rest] = numbers;

console.log(first);
console.log(rest);
```

Output:

```
10
[20, 30, 40, 50]
```

Here:

```
JavaScript
first
```

gets the first value.

And:

```
JavaScript
...rest
```

collects everything remaining.

16. Another Array Destructuring Example

```
JavaScript
const students = [
  "Rahul",
  "Aman",
  "Navi",
  "Priya",
  "Riya"
];

const [first, second, ...others] = students;

console.log(first);
console.log(second);
console.log(others);
```

Output:

```
Rahul
Aman
["Navi", "Priya", "Riya"]
```

17. Rest in Object Destructuring

This is also extremely useful.

```
JavaScript
const user = {
  name: "Navi",
  age: 22,
  city: "Delhi",
  role: "Developer"
};

const { name, ...otherDetails } = user;

console.log(name);
console.log(otherDetails);
```

Output:

```
Navi
{
  age: 22,
  city: "Delhi",
  role: "Developer"
}
```

Here:

```
JavaScript
name
```

extracts one property.

And:

```
JavaScript
...otherDetails
```

collects all remaining properties.

18. Rest vs Spread with Objects

This example makes the difference very clear.

Spread

```
JavaScript
const user = {
  name: "Navi",
  age: 22
};

const newUser = {
  ...user
};
```

Spread **copies/expands** the properties.

Rest

JavaScript

```
const user = {
  name: "Navi",
  age: 22,
  city: "Delhi"
};

const { name, ...details } = user;
```

Rest **collects** the remaining properties.

So:

SPREAD

```
...user
  ↓
name: "Navi"
age: 22
city: "Delhi"
```

Whereas:

REST

```
{ name, ...details }
      ↓
    age + city
```

19. Very Important: Spread Creates Shallow Copies

Consider:

JavaScript

```
const user = {
  name: "Navi",
  address: {
    city: "Delhi"
  }
};

const copy = {
  ...user
};
```

You might think everything is completely copied.

But spread creates a **shallow copy**.

The nested object is still shared.

JavaScript

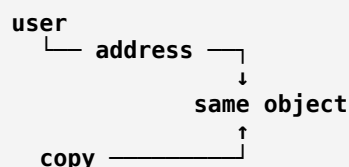
```
copy.address.city = "Mumbai";

console.log(user.address.city);
```

Output:

```
Mumbai
```

Because:



For simple flat objects, spread is excellent.

For deeply nested objects, you need to think about deep copying.

20. Spread with React

If you're learning React, you'll use spread **a lot**.

For example:

```
JavaScript
const [user, setUser] = useState({
  name: "Navi",
  age: 22
});
```

To update only the age:

```
JavaScript
setUser({
  ...user,
  age: 23
});
```

This means:

Keep all existing properties, but change **age**.

Another common example:

```
JavaScript
const [skills, setSkills] = useState([]);

setSkills([
  ...skills,
  "React"
]);
```

If the existing array is:

```
JavaScript
["HTML", "CSS"]
```

then:

```
JavaScript
[
  ...skills,
  "React"
]
```

becomes:

```
JavaScript
["HTML", "CSS", "React"]
```

This pattern is fundamental to React state management.

21. Spread with Function Arguments vs Rest Parameters

These two are almost mirror images.

Spread

```
JavaScript
const numbers = [10, 20, 30];

function add(a, b, c) {
  return a + b + c;
}

add(...numbers);
```

Array → individual arguments.

```
[10, 20, 30]
      ↓
10, 20, 30
```

Rest

```
JavaScript
function add(...numbers) {
  console.log(numbers);
}

add(10, 20, 30);
```

Individual arguments → array.

```
10, 20, 30
      ↓
[10, 20, 30]
```

22. Common Real-World Example

Imagine you have a user object:

```
JavaScript
const user = {
  name: "Navi",
  email: "navi@example.com",
  age: 22,
  role: "Developer"
};
```

You want to remove the `email` from the object:

```
JavaScript
const { email, ...safeUser } = user;
console.log(safeUser);
```

Output:

```
JavaScript
{
  name: "Navi",
  age: 22,
  role: "Developer"
}
```

This is useful when you want to remove sensitive/unwanted properties before passing data somewhere.

23. Another Real-World Example: Shopping Cart

Suppose:

```
JavaScript
const cart = [
  { id: 1, name: "Laptop" },
  { id: 2, name: "Mouse" }
];
```

Add another item without modifying the original array:

```
JavaScript
const newCart = [
  ...cart,
  { id: 3, name: "Keyboard" }
];
```

Result:

```
JavaScript
[
  { id: 1, name: "Laptop" },
  { id: 2, name: "Mouse" },
  { id: 3, name: "Keyboard" }
]
```

This pattern is extremely common in frontend development.

24. Common Mistakes

Mistake 1: Thinking spread and rest are different syntax

They both use:

```
JavaScript
...
```

The **context** determines the meaning.

```
JavaScript
const copy = [...array];    // Spread
function test(...args) {}  // Rest
```

Mistake 2: Using rest in the middle

✗ Invalid:

```
JavaScript
function test(...args, name) {}
```

✓ Correct:

```
JavaScript
function test(name, ...args) {}
```

Mistake 3: Expecting deep copy

```
JavaScript
const copy = {...original};
```

doesn't deeply clone nested objects.

It's a **shallow copy**.

25. Spread vs Rest — Quick Comparison

Feature	Spread	Rest
Syntax	<code>...</code>	<code>...</code>
Purpose	Expand/unpack	Collect/pack
Common use	Arrays/objects	Functions/destructuring
Direction	One → many	Many → one
Example	<code>[...arr]</code>	<code>function f(...args)</code>
Creates array?	Not necessarily	Yes, for rest parameters
Creates object?	Can copy/merge objects	Can collect object properties

26. Easy Way to Remember

Whenever you see ..., ask:

Is it taking something apart?

```
JavaScript
const arr = [1, 2, 3];
console.log(...arr);
```

→ Spread

"Spread these values out."

Is it collecting things together?

```
JavaScript
function test(...args) {
  console.log(args);
}
```

→ Rest

"Collect the remaining values."

The most important examples to remember

Spread Array

```
JavaScript
const a = [1, 2, 3];
const b = [...a];
```

Combine Arrays

```
JavaScript
const result = [...a, ...b];
```

Spread Object

```
JavaScript
const copy = {...user};
```

Update Object

```
JavaScript
const updated = {
  ...user,
  age: 25
};
```

Function Arguments

```
JavaScript
const nums = [10, 20, 30];

Math.max(...nums);
```

Rest Parameters

```
JavaScript
function sum(...numbers) {
  return numbers.reduce((a, b) => a + b, 0);
}
```

Rest Destructuring

```
JavaScript
const [first, ...others] = [10, 20, 30, 40];
```

Object Rest

```
JavaScript
const { password, ...user } = data;
```

In one line:

Spread (...) expands; Rest (...) collects.

If you're learning modern JavaScript for **React + Node.js**, mastering spread/rest along with **destructuring, default parameters, template literals, optional chaining, nullish coalescing, and array methods** will give you a very solid ES6+ foundation.

ChatGPT can make mistakes. Check important info.